

DSA STUDY BLUEPRINT SERIES

Majority Element Blueprint

Boyer-Moore Voting Algorithm Architecture

Section 04 • C++ Core Data Structures & Algorithms

The Majority Element Problem

Finding the element that appears more than $N/2$ times

Brute Force & Hash Maps

- **Brute Force:** Count occurrences of each element using nested loops. $O(N^2)$ time.
- **Hash Map:** Build frequency count of elements. $O(N)$ time but requires **$O(N)$ space**.
- Not optimal when memory is constrained.

Boyer-Moore Voting Algorithm

- Uses voting cancellation strategy.
- Time Complexity: **$O(N)$** .
- Space Complexity: **$O(1)$** (In-place).
- Relies on the fact that the majority element cancels out all other minority votes and still remains.

Algorithmic Core Mechanics

How candidates and counter balancing votes work

Algorithm Phases

Phase 1: Candidate Selection

- Maintain a `candidate` and a `count`.
- If `count == 0`, select current element as candidate.
- If current element == candidate, `count++`. Else, `count--`.

Phase 2: Validation (If required)

- Verify if the candidate actually appears $> N/2$ times by counting its frequency.

Visual Tracing Table

Array: `[2, 2, 1, 1, 1, 2, 2]`

Element	Candidate	Count	Action
2	2	1	Set Candidate, count = 1
2	2	2	Match: count++
1	2	1	Mismatch: count--
1	2	0	Mismatch: count--
1	1	1	Count is 0! Reset candidate to 1
2	1	0	Mismatch: count--
2	2	1	Count is 0! Reset candidate to 2

Boyer-Moore Voting Code

Complete C++ code implementation

```
#include
#include
using namespace std;

int findMajorityElement(vector& nums) {
    int candidate = 0;
    int count = 0;

    // Phase 1: Candidate selection
    for (int num : nums) {
        if (count == 0) {
            candidate = num;
        }
        if (num == candidate) {
            count++;
        } else {
            count--;
        }
    }
    return candidate; // Assuming majority element always exists
}
```